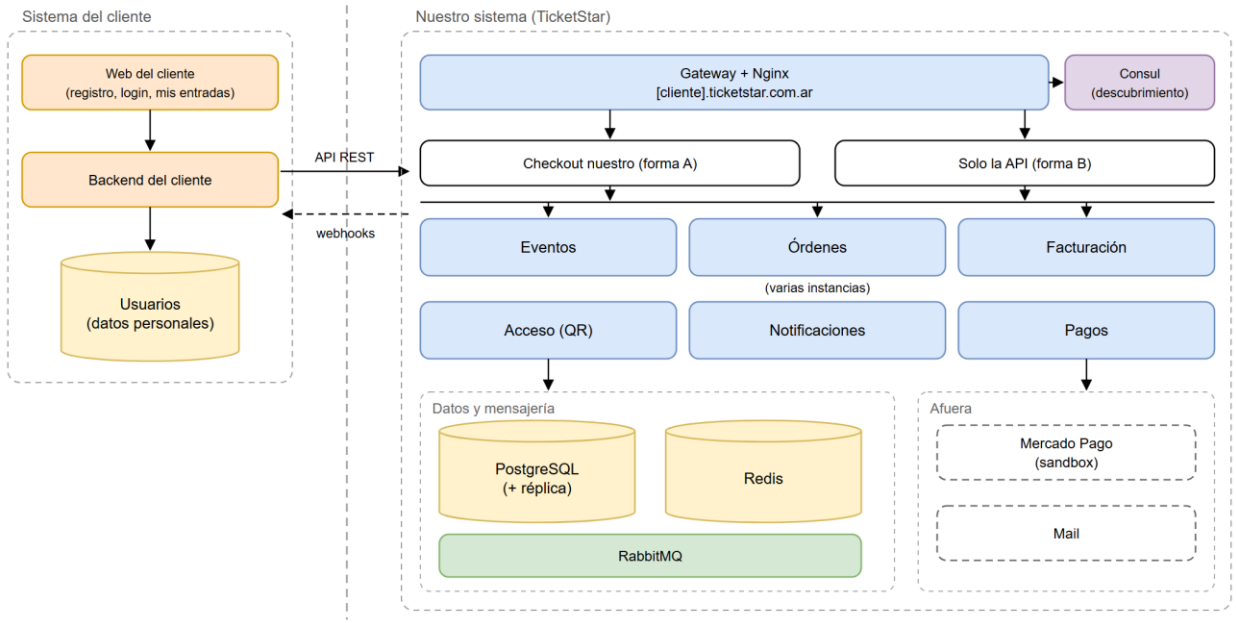
TicketStar

Entrega 1: Propuesta de arquitectura

Informe · Programación Distribuida y Componentes · Proyecto Final

Grupo N°	[completar]
Integrantes	Francisco Aguiar, Rodrigo Rodriguez
Tag / Release	v1-propuesta
Fecha de entrega	[completar]
Repositorio	https://github.com/franaguiargit/ticketstar

En este informe contamos qué problema queremos resolver y cómo pensamos armar el sistema, usando las cinco vistas del modelo 4+1 de Kruchten. Es una propuesta: seguramente algunas cosas cambien cuando empecemos a programar, y si cambian lo vamos a ir actualizando.



Infografía: a la izquierda el sistema del cliente y a la derecha el nuestro.

Las dos flechas que cruzan la línea punteada son toda la conexión entre los dos sistemas: el cliente nos llama por la API y nosotros le avisamos por webhooks cuando pasa algo, por ejemplo cuando se confirma un pago.

1. El problema y para quién es

Cuando sale a la venta un evento grande, en pocos minutos entra muchísima más gente de la que hay entradas. Si el sistema no está preparado se cae o, peor, vende la misma butaca a dos personas. Pasó por ejemplo con la venta de Taylor Swift en Ticketmaster en 2022, donde el sitio colapsó. Después, en la puerta, hay que controlar que una entrada no se use dos veces, por ejemplo si alguien comparte la captura del QR.

Un teatro o un club no tiene por qué resolver todo eso: ya tiene su página y sus clientes, lo que le falta es la parte de la venta. TicketStar es eso, un servicio de venta de entradas que le vendemos a teatros, clubes y productoras, y que se conecta con el sistema de cada uno por una API. Nosotros resolvemos la

fila de espera, la reserva de butacas sin vender dos veces la misma, el pago, la entrada con su QR y el control en la puerta. A cada cliente le damos un subdominio propio, del tipo cliente.ticketstar.com.ar.

Quién usa el sistema

- El cliente (un teatro, un club o una productora): carga sus eventos y conecta su página con nuestra API. Es a quien le vendemos el servicio.
- El público: compra desde la página del cliente, con el usuario que ya tiene ahí, y recibe la entrada por mail. Casi no se entera de que existimos, salvo por un cartel que dice que la venta funciona con TicketStar.
- El personal de acceso: escanea los QR en la puerta del evento desde el celular.

2. Por qué elegimos esta temática

La idea original era hacer el sistema de entradas completo, con los usuarios adentro. Después de hablarlo con el profesor lo replanteamos, porque la materia va más por el lado de conectar sistemas distintos entre sí. Así quedan dos sistemas de dos organizaciones diferentes que se tienen que poner de acuerdo, con un contrato de API en el medio.

Además el tema nos sirve para casi todos los módulos: la apertura de venta es un caso de concurrencia muy claro (dos personas eligiendo la misma butaca al mismo tiempo), es un pico de tráfico para probar la escalabilidad, y tiene seguridad de verdad para pensar, como que un cliente no vea los datos de otro o que no se pueda falsificar una entrada.

El correlato en el mercado de trabajo

No es un problema inventado. En Argentina la venta de entradas la manejan empresas como Ticketek, All Access o Passline, y afuera Ticketmaster o Eventbrite. Eventbrite, por ejemplo, además de vender en su propia página le ofrece a los organizadores una API y un checkout para poner dentro de su sitio, que es bastante parecido a lo que planteamos acá.

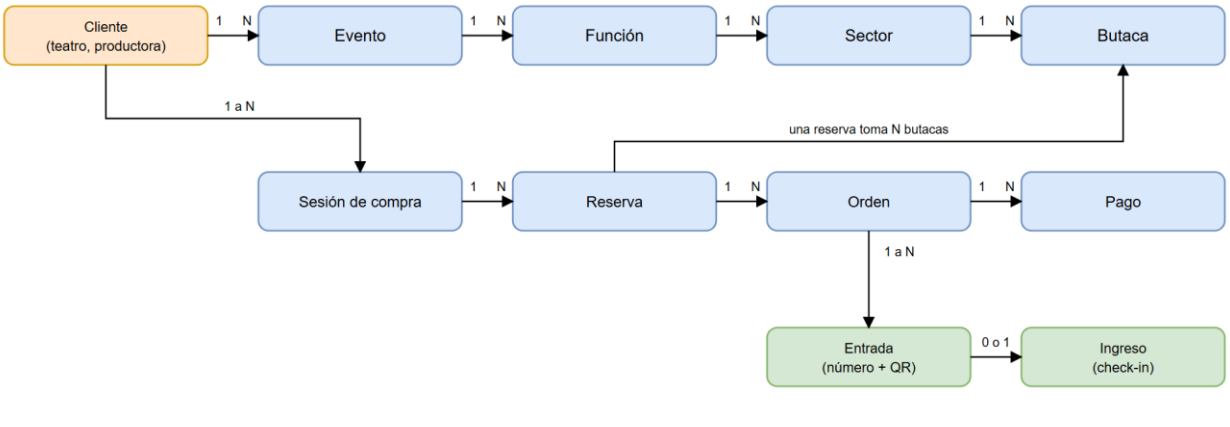
Y la forma de conectarse, una empresa que le ofrece un servicio a otra por una API y le avisa por webhooks cuando pasa algo, es la misma que usan las pasarelas de pago como Mercado Pago o Stripe con los comercios. Integrar sistemas de empresas distintas de esa manera es algo muy común en el trabajo, y es una de las cosas que más nos interesaba aprender con este proyecto.

3. Qué cambió desde la Entrega 0

Nada grande. En la Entrega 0 ya habíamos planteado el sistema conectado con el del cliente; acá lo bajamos a las cuatro vistas más los escenarios, y agregamos qué corre en cada contenedor y cómo se hablan entre ellos. También definimos los lenguajes: nuestros servicios en Python con FastAPI y los clientes de prueba en Node.

4. Vista lógica

Es qué cosas maneja el sistema y cómo se relacionan entre sí.



Entidades principales y sus relaciones.

Las entidades

- Cliente: cada teatro o productora que nos contrata. Es el que separa todo: cada dato del sistema pertenece a un cliente y no se mezcla con el de otro.
- Evento y Función: el evento es por ejemplo una obra, y la función es la fecha y hora concreta. Se vende por función.
- Sector y Butaca: el sector puede ser platea, pullman o campo. Si tiene butacas numeradas, cada una es una fila en la base; si es campo general, lo que hay es un contador de cuántos lugares quedan.
- Sesión de compra: la compra que abre el cliente en nombre de uno de sus usuarios. Dura poco y sirve para que sepamos que esa persona tiene permiso, sin tener que conocerla.
- Reserva: las butacas apartadas por diez minutos mientras la persona paga. No están ni libres ni vendidas.
- Orden y Pago: la compra en sí y el resultado del pago.
- Entrada: lo que se emite cuando el pago se aprueba, con su número y su QR firmado.
- Ingreso: queda registrado cuando la entrada se escanea en la puerta. Una entrada puede entrar una sola vez.

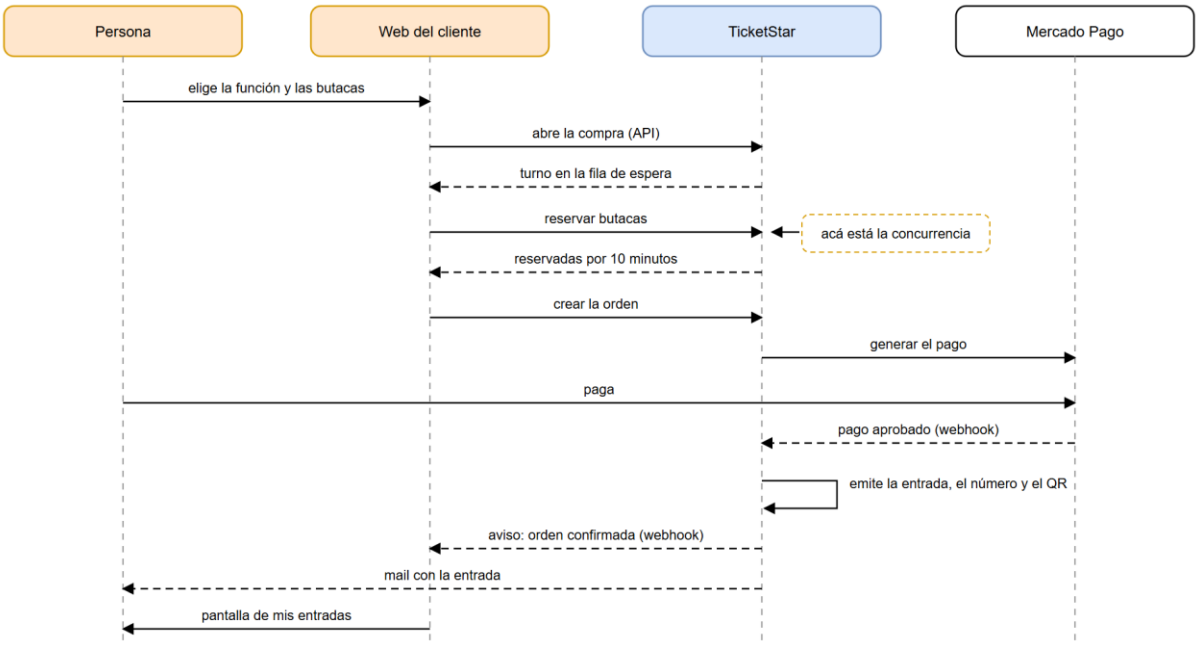
Aparte de esto guardamos, por cada cliente, los avisos que le mandamos y cuántas entradas vendió en el período, que es con lo que después le facturamos.

Lo que no guardamos

Los datos personales de los compradores quedan del lado del cliente. Nosotros guardamos un identificador que nos pasa él y el mail, que lo necesitamos para mandar la entrada. Nos pareció lo más prolijo: son sus clientes, no los nuestros.

5. Vista de procesos

Es qué corre al mismo tiempo y cómo se comunican las partes mientras el sistema está funcionando.



Una compra completa, desde que la persona elige la butaca hasta que ve la entrada.

Qué es síncrono y qué es asíncrono

Momento	Cómo se comunica	Por qué
El cliente reserva una butaca	REST, respuesta en el momento	La persona está esperando en la pantalla y necesita un sí o un no ahora.
El pago se aprueba	Webhook de Mercado Pago hacia nosotros	No sabemos cuándo va a pagar, así que no podemos quedarnos esperando.
Se emite la entrada y se manda el mail	RabbitMQ, entre nuestros servicios	Nadie mira la pantalla esperando el mail, y si el servicio de notificaciones está caído los mensajes quedan en la cola.
Le avisamos al cliente que se confirmó	Webhook nuestro hacia el cliente	Así el cliente no tiene que estar preguntándonos cada dos segundos si ya se pagó.

El punto de concurrencia

Cuando se abre la venta, dos personas pueden elegir la misma butaca en el mismo segundo. La idea es que la reserva sea una sola operación en la base, que cambia la butaca de libre a reservada solamente si en ese momento sigue libre. Postgres bloquea la fila mientras hace el cambio, así que si llegan dos pedidos juntos, el segundo ve que la butaca ya no está libre y no la toca. Al primero le decimos que la butaca es suya y al segundo que se la acaban de llevar, y le mostramos el mapa actualizado.

Lo importante es que no quede un momento entre consultar si está libre y marcarla como reservada donde se pueda colar otro pedido. Para el campo general, en vez de una butaca hay un contador de lugares, y se descuenta con la misma idea: solamente si queda alguno. En la Entrega 3 lo vamos a probar con un script que manda muchos pedidos a la vez por la misma butaca y revisa que se haya vendido una sola vez.

Qué corre en paralelo

- El servicio de órdenes va a tener varias instancias, porque es el que recibe casi toda la carga cuando se abre la venta.
- La fila de espera deja entrar gente de a tandas, así no llegan miles de pedidos juntos a la base.
- El worker de notificaciones va sacando mensajes de la cola a su ritmo.
- El servicio de facturación también escucha la cola y suma las entradas vendidas, pero no participa de la compra: si se cae, nadie deja de comprar.

6. Vista de desarrollo

Es cómo vamos a organizar el código. Pensamos en un solo repositorio con una carpeta por servicio, así es más fácil levantar todo junto con un docker compose up y el que corrige no tiene que clonar diez repos.

```
ticketstar/
  docker-compose.yml
  gateway/            nginx: el subdominio dice de qué cliente es el pedido
  servicios/
    eventos/          eventos, funciones, sectores y butacas
    ordenes/          sesiones de compra, reservas y concurrencia
    pagos/            Mercado Pago y reintentos
    acceso/           validación del QR
    notificaciones/   worker de mails
    facturacion/      consumo y factura del período
  contratos/          la API en OpenAPI
  clientes-demo/
    teatro-avenida/   cliente de prueba, forma A
    club-atletico/    cliente de prueba, forma B
  pruebas/            concurrencia y carga con k6
  docs/               una carpeta por entrega
```

Las dos formas de conectarse

- Forma A: el cliente manda a la persona a una página de checkout nuestra, con su logo, y la recibe de vuelta cuando terminó. Es la más fácil de integrar.
- Forma B: el cliente hace su propia pantalla de compra y solo nos usa por la API. Le da más control pero tiene que programar más.

Las dos usan la misma API; lo único que cambia es quién dibuja la pantalla.

El contrato de la API

La API la definimos primero y recién después programamos. Va documentada con OpenAPI y se puede ver con Swagger. Es la parte más importante del proyecto en cuanto a organización: una vez que un cliente la está usando, si la cambiamos le rompemos el sistema, así que la versionamos (v1) y vamos a poner una prueba que falle si alguien la cambia sin querer.

Los clientes de prueba

Además de nuestro sistema vamos a programar dos páginas que hacen de cliente, una con cada forma de conexión. Las hacemos en Node y no en Python, para mostrar que la conexión anda igual sin importar con qué esté hecho el otro lado.

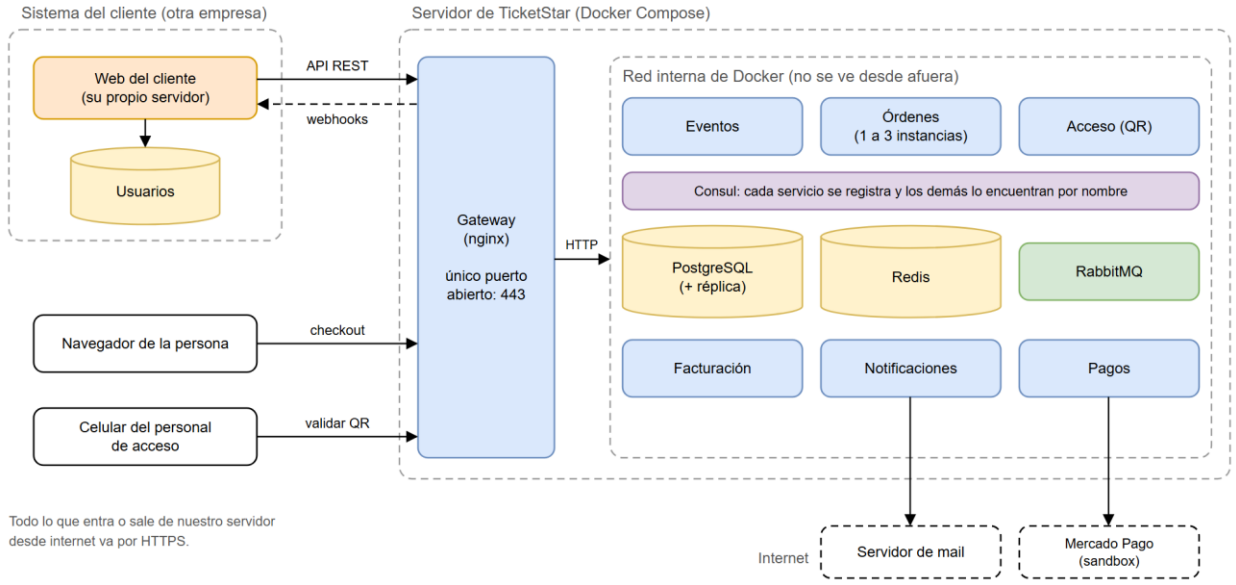
Cómo trabajamos

- Una rama por tarea y un pull request que aprueba el compañero.
- Un tag por entrega, como pide la consigna (v0-plan, v1-propuesta, v2-infraestructura y así).

- El tablero en GitHub Projects, con las tareas de cada sprint.

7. Vista física

Es dónde corre cada cosa. Para la cursada todo corre en una sola máquina con Docker Compose, pero cada contenedor es un nodo aparte que se prende y se apaga por su cuenta. El sistema del cliente corre en otro compose, como si estuviera en el servidor de otra empresa.



Qué corre en cada lado y por dónde se conectan.

Cuántos nodos son y cómo se relacionan

Contenedor	Instancias	Qué expone
gateway (nginx)	1	Lo único que se ve desde afuera. Resuelve a qué cliente corresponde el subdominio y revisa el permiso.
eventos	1	Nada hacia afuera, solo hacia el gateway.
ordenes	1 a 3	Nada hacia afuera. Es el que escalamos cuando se abre la venta.
pagos	1	Recibe el aviso de Mercado Pago, a través del gateway.
acceso	1	Valida los QR en la puerta.
notificaciones	1	No expone nada, solo consume de la cola.
facturacion	1	No expone nada, solo consume de la cola.
postgres	1 + 1 réplica	Solo se ve desde adentro de la red de Docker.
redis	1	Fila de espera y caché.
rabbitmq	1	Los mensajes entre nuestros servicios.
consul	1	Para que los servicios se encuentren por nombre y no por IP.

Son once tipos de contenedor; contando la réplica de la base y las instancias extra de órdenes llegan a unos catorce. Se relacionan así:

- Desde afuera solo se llega al gateway, y todo lo que viene de internet va por HTTPS. El sistema del cliente lo usa para la API, la persona para el checkout y el personal de acceso para validar los QR.
- El gateway reparte los pedidos a los servicios por la red interna de Docker, que desde afuera no se ve.
- Los servicios guardan en Postgres, usan Redis para la fila de espera y se mandan mensajes por RabbitMQ. Consul les permite encontrarse por nombre.

- Pagos y notificaciones son los únicos que salen a internet, para hablar con Mercado Pago y con el servidor de mail.

8. Escenarios (+1)

Estos son los recorridos que atraviesan todas las vistas y que usamos para chequear que la arquitectura cierra.

Escenario principal: comprar cuando se abre la venta

Son las diez de la mañana, se abre la venta de una función con quinientas butacas y hay mucha más gente que entradas.

1. La persona se registra y entra en la página del cliente. Nosotros no participamos.
2. El cliente nos llama para abrir la compra en nombre de su usuario.
3. Le damos un turno en la fila de espera y le vamos diciendo cómo viene.
4. Cuando le toca, pide las butacas. Acá está la concurrencia: si otro se la llevó, le avisamos y elige de nuevo.
5. Se arma la orden y se genera el pago con las credenciales del cliente, así la plata le llega a él.
6. La persona paga. Mercado Pago nos avisa por webhook y nosotros verificamos que el aviso sea de verdad.
7. Se confirma la orden y se emite la entrada con su número y su QR firmado.
8. Le avisamos al cliente por webhook y le mandamos el mail a la persona con la entrada.
9. Cuando la persona entra a sus entradas en la página del cliente, el cliente nos las pide y las muestra.

Cómo pasa este escenario por cada vista

Vista	Qué aparece en este escenario
Lógica	Cliente, Función, Butaca, Sesión de compra, Reserva, Orden, Pago y Entrada.
Procesos	La reserva es REST y espera la respuesta; el aviso de Mercado Pago es un webhook; la emisión de la entrada y el mail van por la cola. Y está el punto de concurrencia cuando dos piden la misma butaca.
Desarrollo	Participan los módulos de órdenes, pagos y notificaciones, más el contrato de la API que usa el cliente para abrir la compra.
Física	El pedido entra por el gateway, lo atiende alguna de las instancias de órdenes, se guarda en Postgres, pasa por RabbitMQ y sale hacia Mercado Pago y el servidor de mail.

Otros escenarios que queremos probar

Escenario	Qué tiene que pasar
Entrar al evento	Varias puertas escanean al mismo tiempo. La primera vez la entrada es válida; la segunda vez dice que ya se usó, con la hora y la puerta.
El cliente no contesta	Si su sistema está caído cuando le queremos avisar, reintentamos varias veces espaciando cada vez más. La compra ya está hecha igual.
Mercado Pago se cae	Después de varios errores seguidos dejamos de llamarlo por un rato y avisamos que el pago no está disponible. El resto del sistema sigue andando.
Se vence la reserva	Si la persona no paga en diez minutos, las butacas vuelven a estar disponibles y le avisamos al cliente.
Dos clientes a la vez	Los dos clientes de prueba funcionando al mismo tiempo, cada uno viendo solo sus propios eventos.

9. Decisiones y riesgos

Decisiones que tomamos

Decisión	Por qué	Qué descartamos
Una sola base con los datos separados por cliente	Dar de alta un cliente nuevo es agregar una fila, no crear una base entera.	Una base por cliente, que aísla más pero nos complica cada cambio.
El cliente cobra con sus propias credenciales	No queremos manejar plata ajena, es un problema legal que no nos corresponde.	Cobrar nosotros y después liquidarle.
Dos formas de conectarse (A y B)	Cubrimos al cliente chico y al grande con la misma API.	Un widget para embeber, que es más trabajo de pantalla y no agrega nada nuevo.
Un solo repositorio	Es más fácil de levantar y de corregir.	Un repositorio por servicio.

Riesgos que vemos

- Es bastante trabajo: además del sistema hay que hacer dos clientes de prueba. Si vamos justos, el segundo cliente lo hacemos más simple.
- La parte de varios clientes puede complicarse más de lo que pensamos, sobre todo que no se mezclen los datos.
- No tenemos experiencia con pruebas de carga, así que k6 lo queremos arrancar antes y no dejarlo para el final.

Lo que todavía tenemos que consultar

- Si los dos clientes de prueba cuentan como parte de la entrega o son solo para mostrar.
- Si el sistema del cliente corriendo aparte sirve como segundo nodo del módulo 2.
- Si una librería para que el cliente se conecte más fácil (un SDK) cuenta como componente del módulo 4.
- Si la pasarela puede ser simulada o hay que usar el sandbox de Mercado Pago.